In [1]:
```python
# Reload dataset with proper column names

import pandas as pd

df = pd.read_csv(
    "twitter_training.csv",
    encoding="latin1",
    header=None,
    names=["id", "topic", "sentiment", "tweet"]
)

print(df.head())
print(df.columns)
```

```
     id        topic sentiment  \
0  2401  Borderlands  Positive
1  2401  Borderlands  Positive
2  2401  Borderlands  Positive
3  2401  Borderlands  Positive
4  2401  Borderlands  Positive

                                               tweet
0  im getting on borderlands and i will murder yo...
1  I am coming to the borders and I will kill you...
2  im getting on borderlands and i will kill you ...
3  im coming on borderlands and i will murder you...
4  im getting on borderlands 2 and i will murder ...
Index(['id', 'topic', 'sentiment', 'tweet'], dtype='object')
```

In [2]:
```python
# ====================================
# TASK 2 : DATA CLEANING
# ====================================

import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download stopwords (run once)
```

```
nltk.download('stopwords')

# Initialize stopwords and stemmer
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()


# ====================================
# 1. CHECK MISSING VALUES
# ====================================

print("Missing values in dataset:")
print(df.isnull().sum())


# ====================================
# 2. HANDLE MISSING VALUES
# ====================================

# Remove rows where tweet text is missing
df.dropna(subset=['tweet'], inplace=True)

print("Shape after removing missing tweets:", df.shape)


# ====================================
# 3. REMOVE DUPLICATES
# ====================================

df.drop_duplicates(subset=['tweet'], inplace=True)

print("Shape after removing duplicates:", df.shape)


# ====================================
# 4. TEXT CLEANING FUNCTION
# ====================================

def clean_text(text):

    text = str(text)
```

```
    # Remove URLs
    text = re.sub(r"http\S+", "", text)

    # Remove mentions
    text = re.sub(r"@\w+", "", text)

    # Remove hashtags
    text = re.sub(r"#\w+", "", text)

    # Remove special characters and numbers
    text = re.sub(r"[^a-zA-Z\s]", "", text)

    # Convert to lowercase
    text = text.lower()

    # Tokenization
    words = text.split()

    # Remove stopwords
    words = [word for word in words if word not in stop_words]

    # Apply stemming
    words = [stemmer.stem(word) for word in words]

    # Join words back into text
    return " ".join(words)


# ====================================
# APPLY TEXT CLEANING
# ====================================

df["clean_tweet"] = df["tweet"].apply(clean_text)

# View cleaned tweets
print(df[['tweet', 'clean_tweet']].head())
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\vashi\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
Missing values in dataset:
id             0
topic          0
sentiment      0
tweet        686
dtype: int64
Shape after removing missing tweets: (73996, 4)
Shape after removing duplicates: (69491, 4)
                                                tweet  \
0  im getting on borderlands and i will murder yo...
1  I am coming to the borders and I will kill you...
2  im getting on borderlands and i will kill you ...
3  im coming on borderlands and i will murder you...
4  im getting on borderlands 2 and i will murder ...

                clean_tweet
0   im get borderland murder
1           come border kill
2     im get borderland kill
3  im come borderland murder
4   im get borderland murder
```

In [3]:

```python
# ====================================
# TASK 3 : FEATURE ENGINEERING
# ====================================

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical


# ====================================
# 1. TOKENIZATION
# ====================================

# Maximum vocabulary size
max_words = 10000

# Maximum tweet length
max_len = 50
```

```python
# Create tokenizer
tokenizer = Tokenizer(num_words=max_words)

# Fit tokenizer on cleaned tweets
tokenizer.fit_on_texts(df["clean_tweet"])


# ======================================
# 2. CONVERT TEXT TO SEQUENCES
# ======================================

# Convert tweets into sequences of integers
sequences = tokenizer.texts_to_sequences(df["clean_tweet"])


# ======================================
# 3. PAD SEQUENCES
# ======================================

# Ensure all tweets have same length
X = pad_sequences(sequences, maxlen=max_len)


# ======================================
# 4. ENCODE SENTIMENT LABELS
# ======================================

# Convert sentiment labels into numbers
label_map = {
    "Positive":0,
    "Negative":1,
    "Neutral":2,
    "Irrelevant":2
}

df["label"] = df["sentiment"].map(label_map)

# Convert labels to categorical format
y = to_categorical(df["label"], num_classes=3)
```

```
# ====================================
# CHECK RESULT
# ====================================

print("Feature matrix shape:", X.shape)
print("Label shape:", y.shape)
```

```
Feature matrix shape: (69491, 50)
Label shape: (69491, 3)
```

In [4]:

```
# ====================================
# PART 2 : EXPLORATORY DATA ANALYSIS
# BASIC STATISTICS
# ====================================

import pandas as pd

# ------------------------------------
# DATASET OVERVIEW
# ------------------------------------

print("Dataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nFirst 5 rows:")
print(df.head())


# ------------------------------------
# SUMMARY STATISTICS
# ------------------------------------

print("\nSummary Statistics:")
print(df.describe(include='all'))


# ------------------------------------
# MEAN / MEDIAN / MODE OF TWEET LENGTH
# ------------------------------------
```

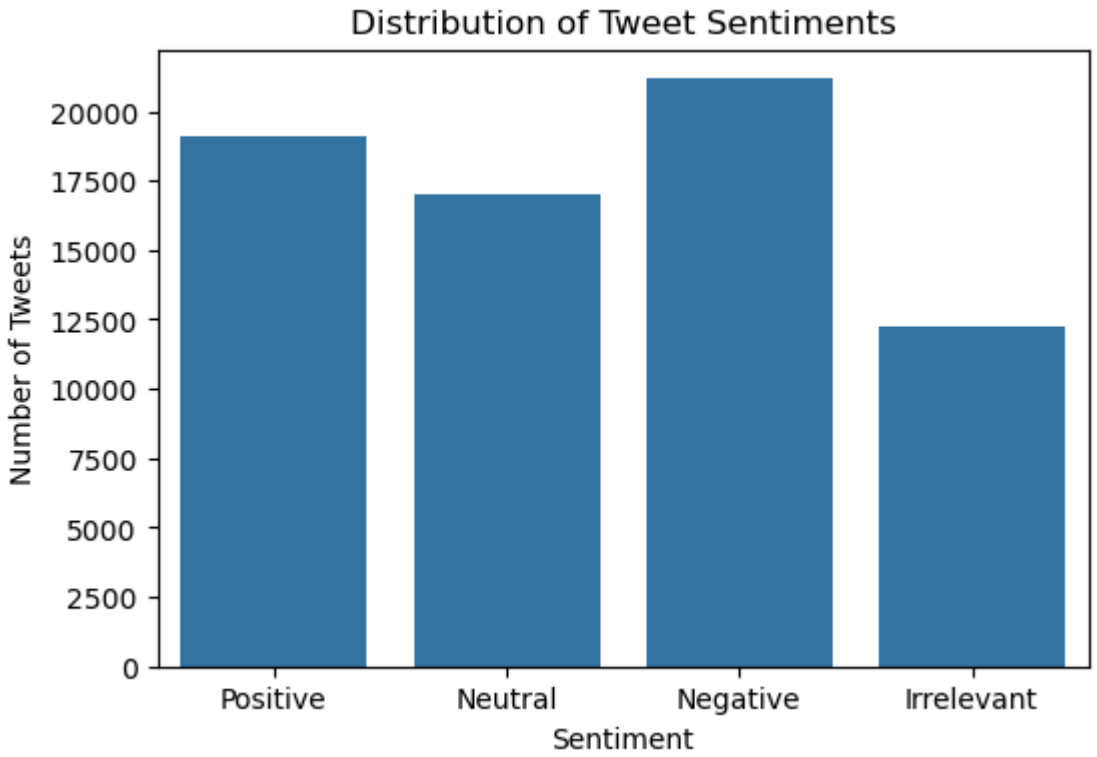
```python
# Create tweet length feature
df["tweet_length"] = df["clean_tweet"].apply(len)

print("\nMean Tweet Length:", df["tweet_length"].mean())
print("Median Tweet Length:", df["tweet_length"].median())
print("Mode Tweet Length:", df["tweet_length"].mode()[0])


# ---------------------------------------
# SENTIMENT DISTRIBUTION
# ---------------------------------------

sentiment_counts = df["sentiment"].value_counts()

print("\nSentiment Distribution:")
print(sentiment_counts)


# Separate counts for clarity
positive_count = sentiment_counts.get("Positive", 0)
negative_count = sentiment_counts.get("Negative", 0)
neutral_count = sentiment_counts.get("Neutral", 0)

print("\nNumber of Positive Tweets:", positive_count)
print("Number of Negative Tweets:", negative_count)
print("Number of Neutral Tweets:", neutral_count)
```

```
Dataset Shape: (69491, 6)

Column Names:
Index(['id', 'topic', 'sentiment', 'tweet', 'clean_tweet', 'label'], dtype='object')

First 5 rows:
     id        topic sentiment  \
0  2401  Borderlands  Positive
1  2401  Borderlands  Positive
2  2401  Borderlands  Positive
3  2401  Borderlands  Positive
4  2401  Borderlands  Positive

                                               tweet  \
0  im getting on borderlands and i will murder yo...
1  I am coming to the borders and I will kill you...
2  im getting on borderlands and i will kill you ...
3  im coming on borderlands and i will murder you...
4  im getting on borderlands 2 and i will murder ...

                clean_tweet  label
0   im get borderland murder      0
1           come border kill      0
2     im get borderland kill      0
3  im come borderland murder      0
4   im get borderland murder      0

Summary Statistics:
                  id      topic sentiment  \
count   69491.000000      69491     69491
unique           NaN         32         4
top              NaN  MaddenNFL  Negative
freq             NaN       2260     21166
mean     6421.894663        NaN       NaN
std      3741.234004        NaN       NaN
min         1.000000        NaN       NaN
25%      3176.000000        NaN       NaN
50%      6406.000000        NaN       NaN
75%      9584.000000        NaN       NaN
max     13200.000000        NaN       NaN
```

```
                                                    tweet clean_tweet  \
count                                               69491       69491
unique                                              69491       61704
top     Just like the windows partition of my Mac is l...
freq                                                    1         377
mean                                                  NaN         NaN
std                                                   NaN         NaN
min                                                   NaN         NaN
25%                                                   NaN         NaN
50%                                                   NaN         NaN
75%                                                   NaN         NaN
max                                                   NaN         NaN

               label
count   69491.000000
unique           NaN
top              NaN
freq             NaN
mean        1.146652
std         0.820924
min         0.000000
25%         0.000000
50%         1.000000
75%         2.000000
max         2.000000

Mean Tweet Length: 65.76419968053418
Median Tweet Length: 56.0
Mode Tweet Length: 29

Sentiment Distribution:
sentiment
Negative      21166
Positive      19067
Neutral       17042
Irrelevant    12216
Name: count, dtype: int64

Number of Positive Tweets: 19067
Number of Negative Tweets: 21166
Number of Neutral Tweets: 17042
```

In [5]:
```
# ====================================
# IMPORT LIBRARIES FOR VISUALIZATION
# ====================================

import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from wordcloud import WordCloud
```

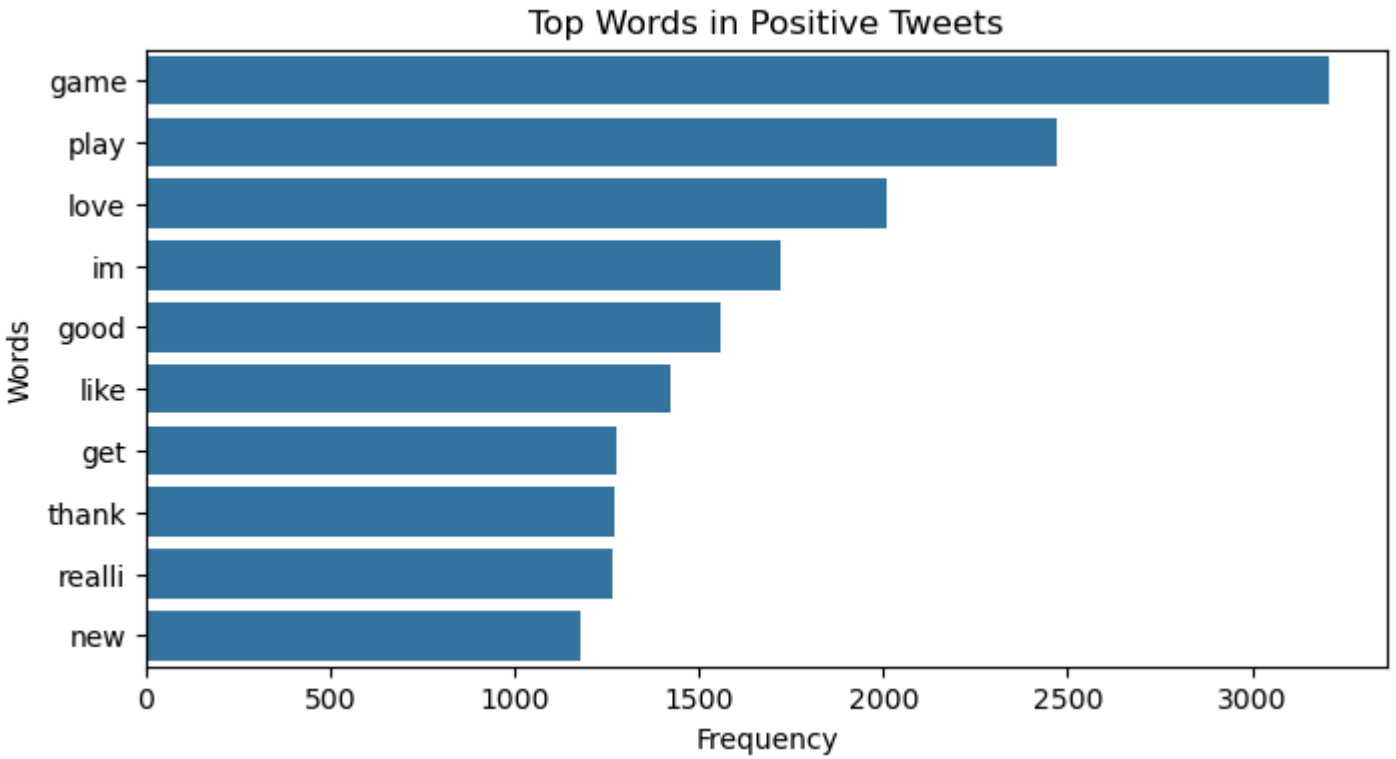
In [6]:
```
# ====================================
# SENTIMENT DISTRIBUTION
# ====================================

plt.figure(figsize=(6,4))

sns.countplot(x='sentiment', data=df)

plt.title("Distribution of Tweet Sentiments")
plt.xlabel("Sentiment")
plt.ylabel("Number of Tweets")

plt.show()
```

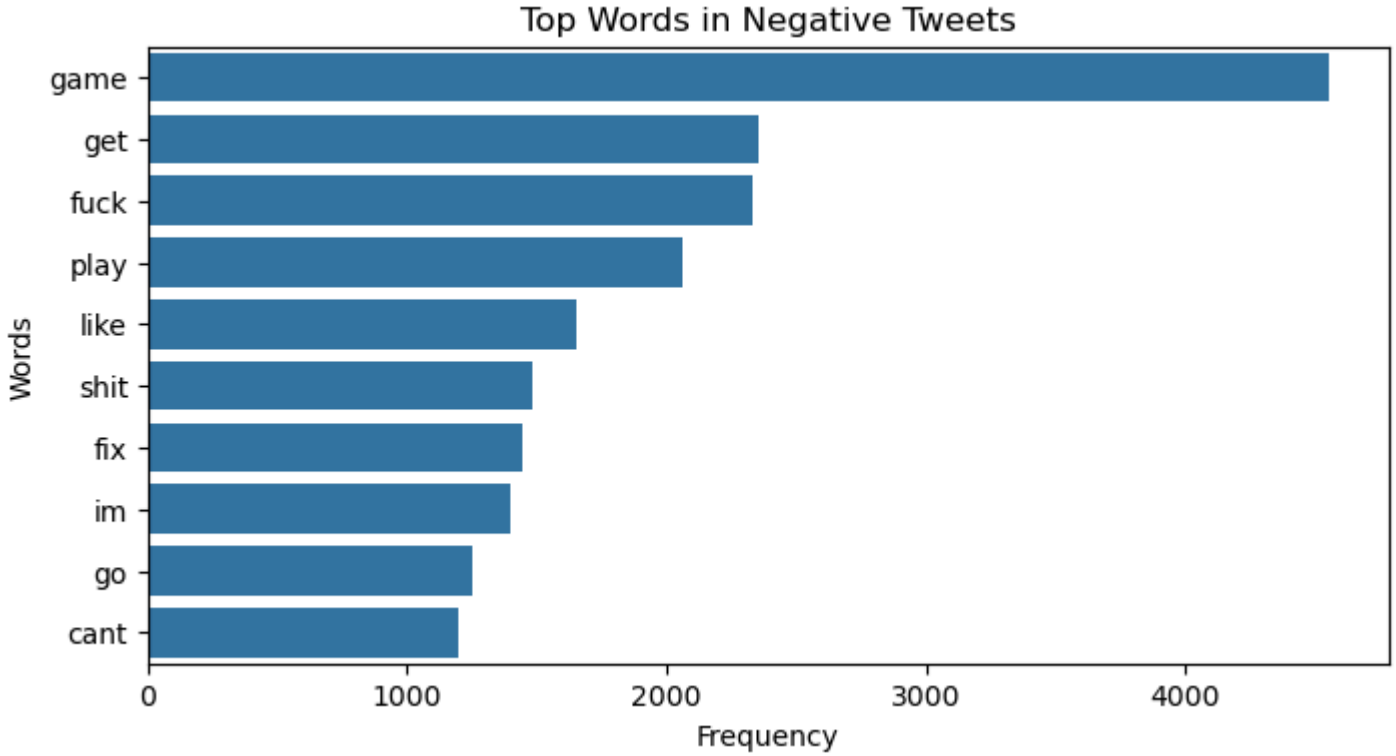


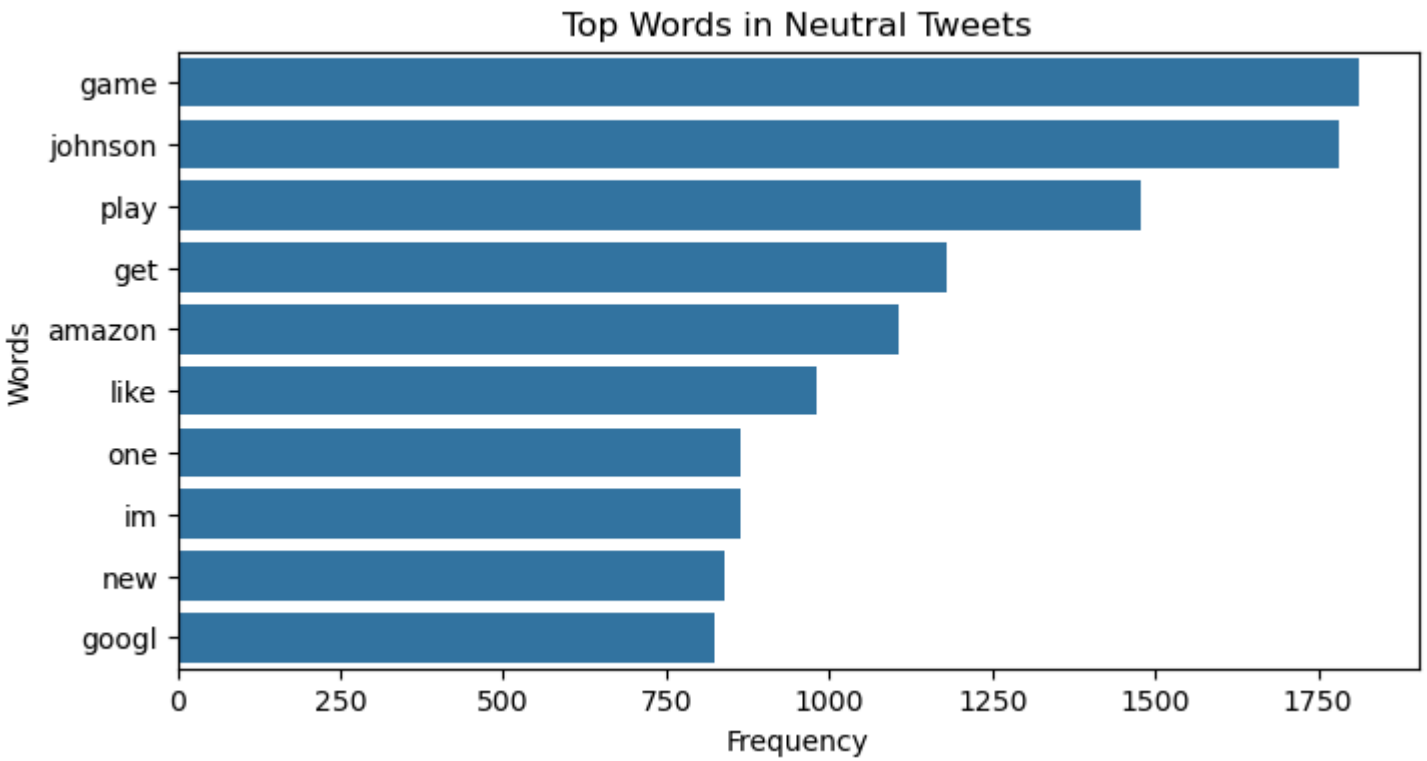
In [7]:

```
# ====================================
# TOP WORDS IN EACH SENTIMENT
# ====================================

def plot_top_words(sentiment_label):

    text = " ".join(df[df['sentiment'] == sentiment_label]['clean_tweet'])

    words = text.split()

    word_freq = Counter(words)

    common_words = word_freq.most_common(10)

    words = [w[0] for w in common_words]
    counts = [w[1] for w in common_words]
```

```
    plt.figure(figsize=(8,4))

    sns.barplot(x=counts, y=words)

    plt.title(f"Top Words in {sentiment_label} Tweets")

    plt.xlabel("Frequency")
    plt.ylabel("Words")

    plt.show()


# Plot for each sentiment
plot_top_words("Positive")
plot_top_words("Negative")
plot_top_words("Neutral")
```




Top Words in Negative Tweets
game
get
fuck
play
like
shit
fix
im
go
cant
Words
0
1000
2000
3000
4000
Frequency

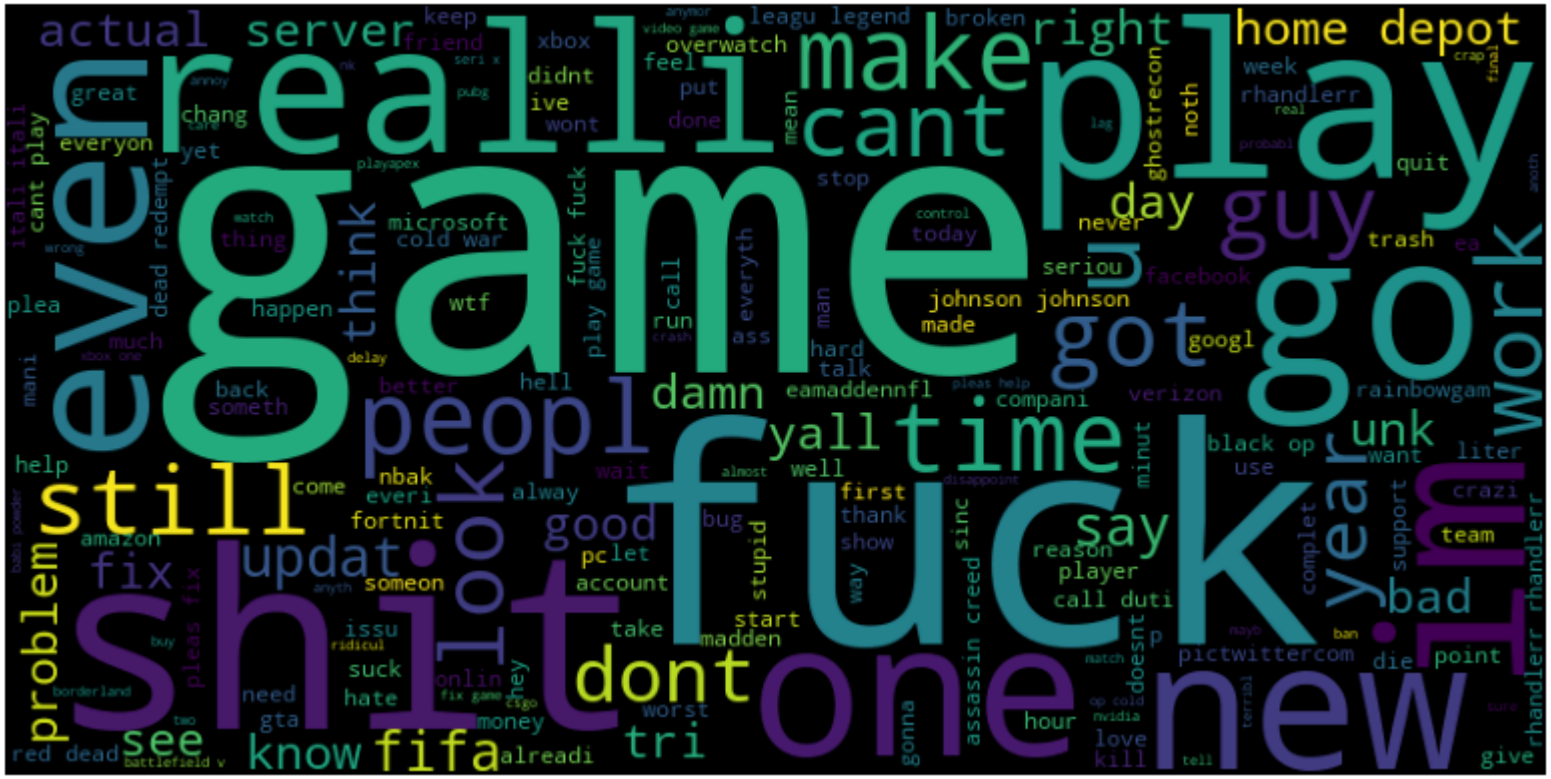


In [8]:

```
# ====================================
# WORD CLOUD FOR POSITIVE TWEETS
# ====================================

positive_text = " ".join(df[df['sentiment']=="Positive"]['clean_tweet'])

wordcloud = WordCloud(width=800, height=400, background_color='white').generate(positive_text)

plt.figure(figsize=(10,5))
plt.imshow(wordcloud)
plt.axis("off")
plt.title("Word Cloud - Positive Tweets")
plt.show()


# ====================================
```

```
# WORD CLOUD FOR NEGATIVE TWEETS
# ===================================

negative_text = " ".join(df[df['sentiment']=="Negative"]['clean_tweet'])

wordcloud = WordCloud(width=800, height=400, background_color='black').generate(negative_text)

plt.figure(figsize=(10,5))
plt.imshow(wordcloud)
plt.axis("off")
plt.title("Word Cloud - Negative Tweets")
plt.show()
```

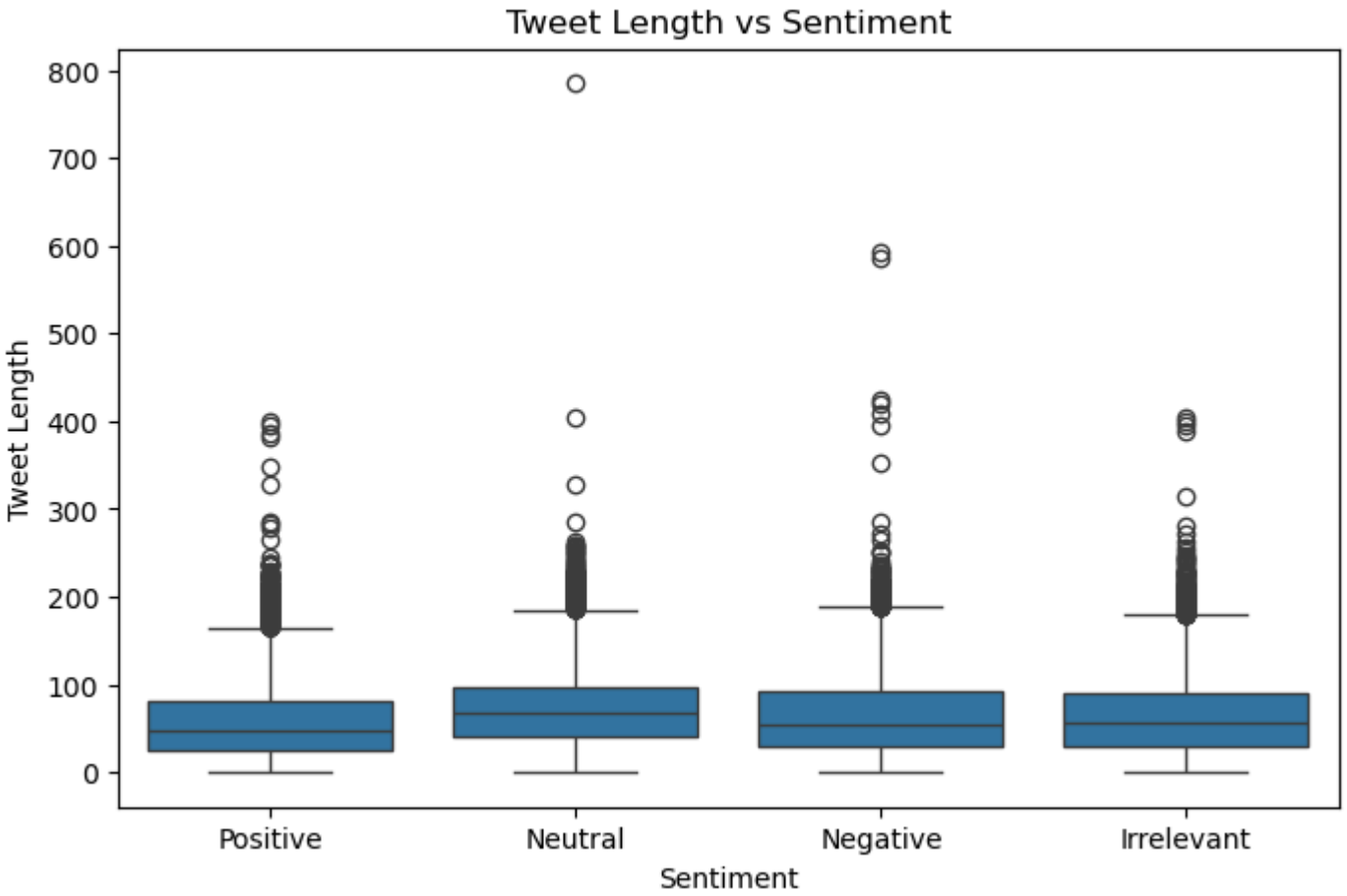
Word Cloud - Negative Tweets



In [9]:

```
# ======================================
# RELATIONSHIP BETWEEN TWEET LENGTH AND SENTIMENT
# ======================================

# Create tweet length column if not already created
df["tweet_length"] = df["clean_tweet"].apply(len)

plt.figure(figsize=(8,5))

sns.boxplot(x="sentiment", y="tweet_length", data=df)

plt.title("Tweet Length vs Sentiment")

plt.xlabel("Sentiment")
```

```
plt.ylabel("Tweet Length")

plt.show()
```



In [10]:
```
# ====================================
# PART 2 : INSIGHTS FROM EDA
# ====================================

print("\n" + "="*70)
print("INSIGHTS FROM EXPLORATORY DATA ANALYSIS (EDA)")
print("="*70)
```

```
print("\n1. Sentiment Distribution")
print("The sentiment distribution visualization shows how tweets are divided")
print("into positive, negative, and neutral categories. The plot helps in")
print("understanding whether the dataset is balanced or dominated by a")
print("particular sentiment class. If one sentiment appears significantly")
print("more frequently than others, it may influence the model's learning.")

print("\n2. Word Frequency Patterns")
print("The frequency analysis of words in each sentiment category reveals")
print("clear differences in language usage. Positive tweets generally")
print("contain words related to appreciation, enjoyment, or satisfaction,")
print("while negative tweets contain words expressing frustration or")
print("criticism. Neutral tweets usually contain descriptive or factual")
print("statements without strong emotional tone.")

print("\n3. Word Cloud Observations")
print("Word clouds for positive and negative tweets highlight the most")
print("frequently used terms in each sentiment category. These visual")
print("representations help quickly identify dominant keywords associated")
print("with positive or negative emotions in the dataset.")

print("\n4. Tweet Length vs Sentiment")
print("The relationship between tweet length and sentiment suggests that")
print("tweet size varies across sentiment categories. In some cases,")
print("negative tweets appear slightly longer because users may provide")
print("more detailed explanations when expressing complaints.")

print("\nOverall Conclusion")
print("The EDA results show that word usage patterns and tweet structure")
print("are closely related to sentiment categories. These patterns confirm")
print("that machine learning and deep learning models, such as RNNs, can")
print("effectively learn from textual data to perform sentiment classification.")
```

```
======================================================================
INSIGHTS FROM EXPLORATORY DATA ANALYSIS (EDA)
======================================================================

1. Sentiment Distribution
The sentiment distribution visualization shows how tweets are divided
into positive, negative, and neutral categories. The plot helps in
understanding whether the dataset is balanced or dominated by a
particular sentiment class. If one sentiment appears significantly
more frequently than others, it may influence the model's learning.

2. Word Frequency Patterns
The frequency analysis of words in each sentiment category reveals
clear differences in language usage. Positive tweets generally
contain words related to appreciation, enjoyment, or satisfaction,
while negative tweets contain words expressing frustration or
criticism. Neutral tweets usually contain descriptive or factual
statements without strong emotional tone.

3. Word Cloud Observations
Word clouds for positive and negative tweets highlight the most
frequently used terms in each sentiment category. These visual
representations help quickly identify dominant keywords associated
with positive or negative emotions in the dataset.

4. Tweet Length vs Sentiment
The relationship between tweet length and sentiment suggests that
tweet size varies across sentiment categories. In some cases,
negative tweets appear slightly longer because users may provide
more detailed explanations when expressing complaints.

Overall Conclusion
The EDA results show that word usage patterns and tweet structure
are closely related to sentiment categories. These patterns confirm
that machine learning and deep learning models, such as RNNs, can
effectively learn from textual data to perform sentiment classification.
```

In [11]:
```
# ====================================
# PART 3 : RNN MODEL ARCHITECTURE
# ====================================
```

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

print("\n" + "="*60)
print("BUILDING RNN MODEL ARCHITECTURE (LSTM)")
print("="*60)


# Parameters
vocab_size = 10000
embedding_dim = 128
max_len = 50


# Build Model
model = Sequential()

# Embedding Layer (Input shape FIX)
model.add(Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    input_shape=(max_len,)
))

# LSTM Layer
model.add(LSTM(128))

# Dropout
model.add(Dropout(0.3))

# Hidden Dense Layer
model.add(Dense(64, activation='relu'))

# Output Layer (3 sentiments)
model.add(Dense(3, activation='softmax'))


# Compile Model
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
```

```
    metrics=['accuracy']
)


# Show model summary
print("\nModel Architecture Summary:\n")
model.summary()
```

```
============================================================
BUILDING RNN MODEL ARCHITECTURE (LSTM)
============================================================
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model Architecture Summary:

C:\ProgramData\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:103: UserWarning: Do not pass an `input_shape`/`i
nput_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the mo
del instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 50, 128)	1,280,000
lstm (LSTM)	(None, 128)	131,584
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dense_1 (Dense)	(None, 3)	195

Total params: 1,420,035 (5.42 MB)

Trainable params: 1,420,035 (5.42 MB)

Non-trainable params: 0 (0.00 B)

In [12]:

```
# ====================================
# PART 3 : MODEL IMPLEMENTATION
# ====================================

from sklearn.model_selection import train_test_split
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout


print("\n" + "="*70)
print("MODEL IMPLEMENTATION : TRAIN / TEST SPLIT + TRAINING")
print("="*70)


# ------------------------------------------------
# 1. TRAIN TEST SPLIT
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("\nTraining samples:", X_train.shape)
print("Testing samples:", X_test.shape)


# ------------------------------------------------
# 2. BUILD RNN MODEL (WITH DROPOUT + BATCH NORM)
# ------------------------------------------------

vocab_size = 10000
embedding_dim = 128
max_len = 50

model = Sequential()

# Embedding Layer
```

```
model.add(Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    input_shape=(max_len,)
))

# LSTM Layer
model.add(LSTM(128))

# Batch Normalization
model.add(BatchNormalization())

# Dropout
model.add(Dropout(0.3))

# Dense Hidden Layer
model.add(Dense(64, activation='relu'))

# Output Layer
model.add(Dense(3, activation='softmax'))


# Compile Model
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

print("\nModel Architecture:\n")
model.summary()


# ------------------------------------------------
# 3. TRAIN MODEL
# ------------------------------------------------

history = model.fit(
    X_train,
    y_train,
    epochs=5,
```

```
    batch_size=64,
    validation_data=(X_test, y_test)
)


# ------------------------------------------------
# 4. EVALUATE MODEL
# ------------------------------------------------

loss, accuracy = model.evaluate(X_test, y_test)

print("\n" + "="*50)
print("MODEL PERFORMANCE ON TEST DATA")
print("="*50)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
======================================================================
MODEL IMPLEMENTATION : TRAIN / TEST SPLIT + TRAINING
======================================================================

Training samples: (55592, 50)
Testing samples: (13899, 50)

Model Architecture:
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 50, 128)	1,280,000
lstm_1 (LSTM)	(None, 128)	131,584
batch_normalization (BatchNormalization)	(None, 128)	512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 3)	195

Total params: 1,420,547 (5.42 MB)

Trainable params: 1,420,291 (5.42 MB)

Non-trainable params: 256 (1.00 KB)

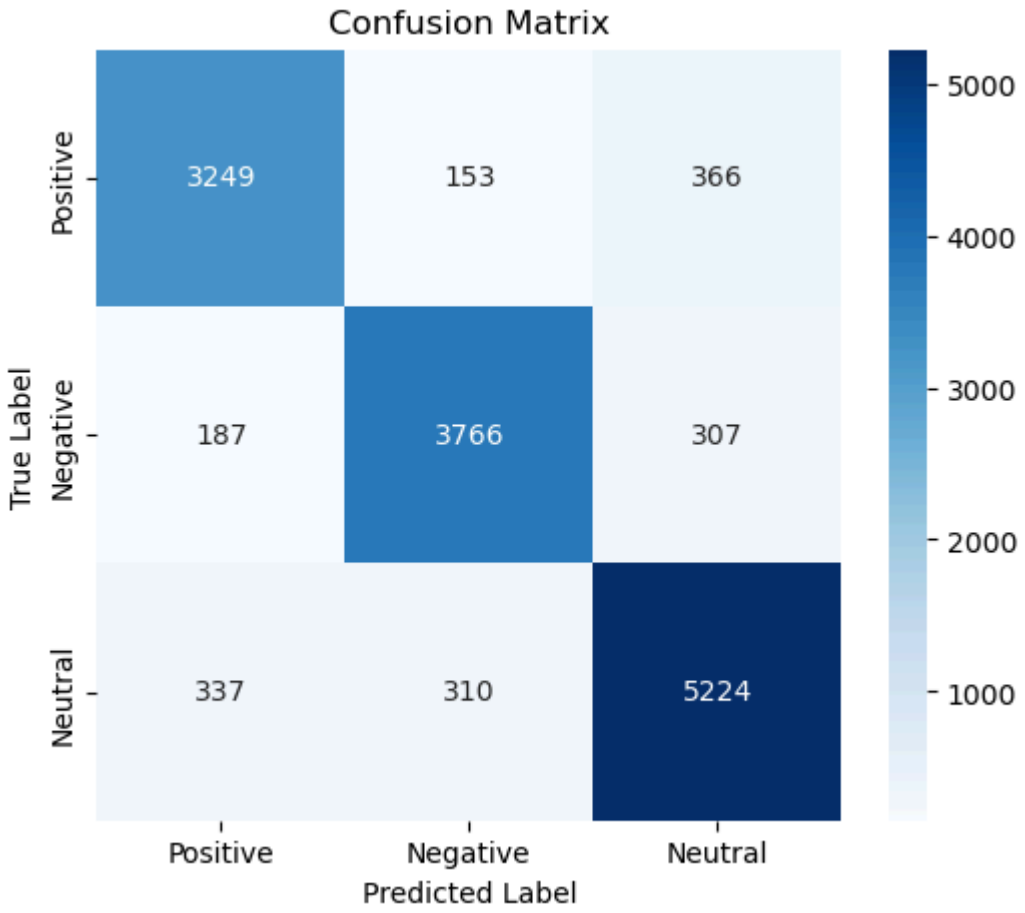
```
Epoch 1/5
869/869 ━━━━━━━━━━━━━━━━━━━━ 62s 67ms/step - accuracy: 0.6953 - loss: 0.6906 - val_accuracy: 0.7966 - val_loss: 0.5093
Epoch 2/5
869/869 ━━━━━━━━━━━━━━━━━━━━ 56s 65ms/step - accuracy: 0.8493 - loss: 0.3812 - val_accuracy: 0.8014 - val_loss: 0.5027
Epoch 3/5
869/869 ━━━━━━━━━━━━━━━━━━━━ 82s 64ms/step - accuracy: 0.8988 - loss: 0.2633 - val_accuracy: 0.8552 - val_loss: 0.4049
Epoch 4/5
869/869 ━━━━━━━━━━━━━━━━━━━━ 61s 71ms/step - accuracy: 0.9180 - loss: 0.2107 - val_accuracy: 0.8621 - val_loss: 0.4036
Epoch 5/5
869/869 ━━━━━━━━━━━━━━━━━━━━ 79s 67ms/step - accuracy: 0.9332 - loss: 0.1729 - val_accuracy: 0.8806 - val_loss: 0.3671
435/435 ━━━━━━━━━━━━━━━━━━━━ 8s 17ms/step - accuracy: 0.8806 - loss: 0.3671

==================================================
MODEL PERFORMANCE ON TEST DATA
==================================================
Test Loss: 0.36707237362861633
Test Accuracy: 0.8805669546127319
```

In [13]:
```
# ====================================
# PART 3 : MODEL EVALUATION
```

```python
# ====================================

from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

print("\n" + "="*60)
print("MODEL EVALUATION")
print("="*60)


# ----------------------------------------
# 1. PREDICTIONS
# ----------------------------------------

y_pred_prob = model.predict(X_test)

# convert probabilities to class labels
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y_test, axis=1)


# ----------------------------------------
# 2. CLASSIFICATION METRICS
# ----------------------------------------

print("\nClassification Report:\n")

print(classification_report(
    y_true,
    y_pred,
    target_names=["Positive","Negative","Neutral"]
))


# ----------------------------------------
# 3. CONFUSION MATRIX
# ----------------------------------------

cm = confusion_matrix(y_true, y_pred)
```

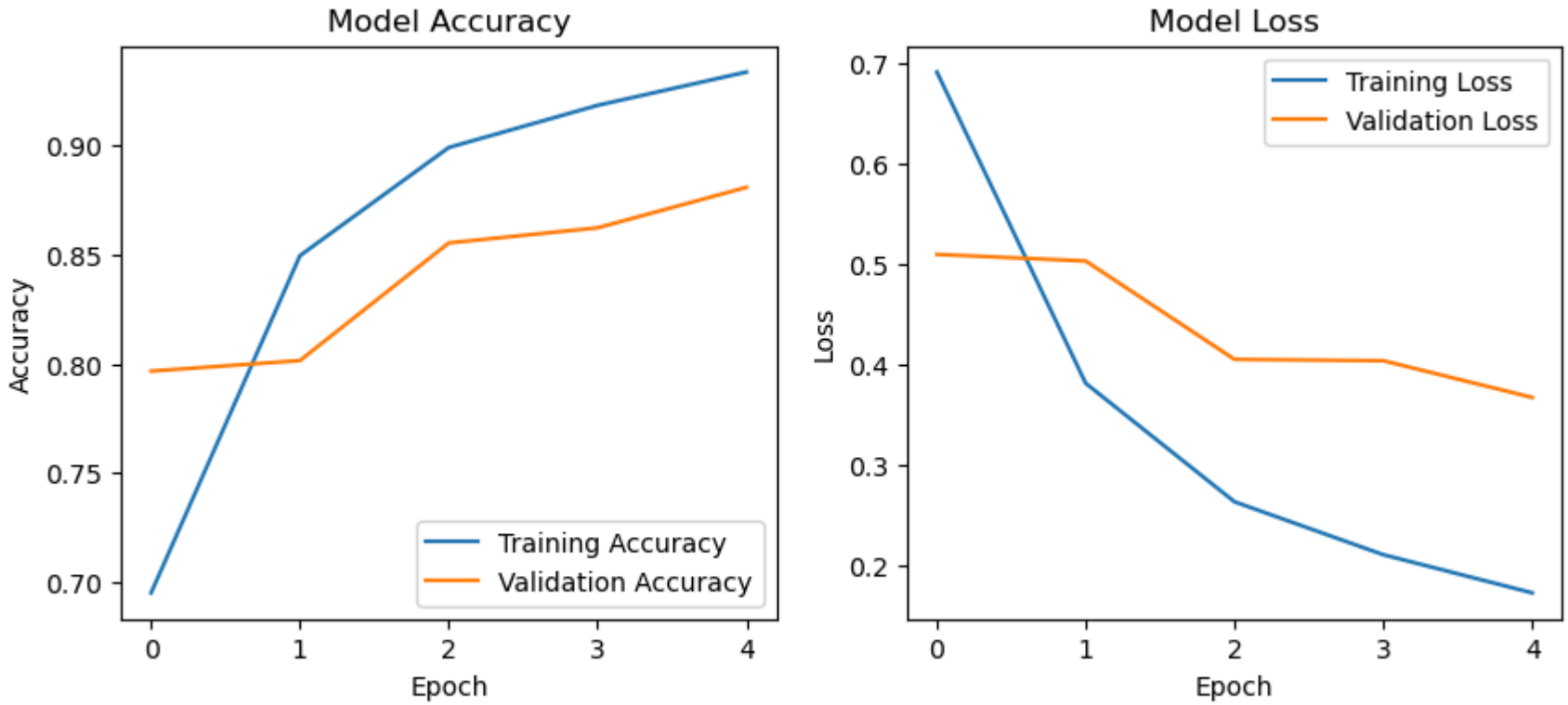
```
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Positive","Negative","Neutral"],
    yticklabels=["Positive","Negative","Neutral"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.show()
```

```
============================================================
MODEL EVALUATION
============================================================
435/435 ━━━━━━━━━━━━━━━━━━━━ 8s 17ms/step

Classification Report:

              precision    recall  f1-score   support

    Positive       0.86      0.86      0.86      3768
    Negative       0.89      0.88      0.89      4260
     Neutral       0.89      0.89      0.89      5871

    accuracy                           0.88     13899
   macro avg       0.88      0.88      0.88     13899
weighted avg       0.88      0.88      0.88     13899
```



In [14]:
```
# ====================================
# LEARNING CURVES
# ====================================

plt.figure(figsize=(10,4))

# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
plt.legend()


# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()
```



In [15]:
```
# ====================================
# HYPERPARAMETER TUNING
```

```
# ====================================

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Input

def build_model(lstm_units=64, dropout_rate=0.3):

    model = Sequential([
        Input(shape=(50,)),          # Correct input layer

        Embedding(
            input_dim=10000,
            output_dim=128
        ),

        LSTM(lstm_units),

        Dropout(dropout_rate),

        Dense(64, activation='relu'),

        Dense(3, activation='softmax')
    ])

    model.compile(
        loss='categorical_crossentropy',
        optimizer='adam',
        metrics=['accuracy']
    )

    return model


# Hyperparameter options
lstm_options = [64, 128]
dropout_options = [0.2, 0.3]

best_accuracy = 0
best_params = None
```

```
for lstm_units in lstm_options:
    for dropout_rate in dropout_options:

        print("\nTesting LSTM:", lstm_units,
              " Dropout:", dropout_rate)

        model = build_model(lstm_units, dropout_rate)

        model.fit(
            X_train,
            y_train,
            epochs=3,
            batch_size=64,
            validation_data=(X_test, y_test),
            verbose=0
        )

        loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

        print("Accuracy:", accuracy)

        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_params = (lstm_units, dropout_rate)


print("\nBest Parameters:")
print("LSTM Units:", best_params[0])
print("Dropout:", best_params[1])
print("Best Accuracy:", best_accuracy)
```

```
Testing LSTM: 64  Dropout: 0.2
Accuracy: 0.8373983502388

Testing LSTM: 64  Dropout: 0.3
Accuracy: 0.8381897807121277

Testing LSTM: 128  Dropout: 0.2
Accuracy: 0.8403482437133789

Testing LSTM: 128  Dropout: 0.3
Accuracy: 0.8384056687355042

Best Parameters:
LSTM Units: 128
Dropout: 0.2
Best Accuracy: 0.8403482437133789
```

In [16]:

```python
# ====================================
# PART 3 : MODEL IMPROVEMENT TECHNIQUES
# ====================================

from sklearn.model_selection import ParameterGrid, KFold
import numpy as np
import tensorflow as tf

print("\n" + "="*70)
print("MODEL IMPROVEMENT USING GRID SEARCH AND CROSS VALIDATION")
print("="*70)


# ----------------------------------------------------
# GRID SEARCH FOR BEST HYPERPARAMETERS
# ----------------------------------------------------

param_grid = {
    "lstm_units": [64, 128],
    "dropout_rate": [0.2, 0.3],
    "learning_rate": [0.001, 0.0005]
}

grid = list(ParameterGrid(param_grid))
```

```
best_score = 0
best_params = None


for params in grid:

    print("\nTesting Parameters:", params)

    model = Sequential([
        Input(shape=(50,)),
        Embedding(input_dim=10000, output_dim=128),
        LSTM(params["lstm_units"]),
        Dropout(params["dropout_rate"]),
        Dense(64, activation="relu"),
        Dense(3, activation="softmax")
    ])

    optimizer = tf.keras.optimizers.Adam(
        learning_rate=params["learning_rate"]
    )

    model.compile(
        loss="categorical_crossentropy",
        optimizer=optimizer,
        metrics=["accuracy"]
    )

    model.fit(
        X_train,
        y_train,
        epochs=3,
        batch_size=64,
        verbose=0
    )

    loss, acc = model.evaluate(X_test, y_test, verbose=0)

    print("Accuracy:", acc)

    if acc > best_score:
```

```
        best_score = acc
        best_params = params


print("\nBest Hyperparameters Found:")
print(best_params)
print("Best Accuracy:", best_score)

# ====================================
# CROSS VALIDATION
# ====================================

kfold = KFold(n_splits=3, shuffle=True, random_state=42)

cv_scores = []

for train_idx, val_idx in kfold.split(X):

    X_train_cv, X_val_cv = X[train_idx], X[val_idx]
    y_train_cv, y_val_cv = y[train_idx], y[val_idx]

    model = Sequential([
        Input(shape=(50,)),
        Embedding(input_dim=10000, output_dim=128),
        LSTM(128),
        Dropout(0.3),
        Dense(64, activation="relu"),
        Dense(3, activation="softmax")
    ])

    model.compile(
        loss="categorical_crossentropy",
        optimizer="adam",
        metrics=["accuracy"]
    )

    model.fit(
        X_train_cv,
        y_train_cv,
        epochs=3,
        batch_size=64,
```

```
        verbose=0
    )

    loss, acc = model.evaluate(X_val_cv, y_val_cv, verbose=0)

    cv_scores.append(acc)


print("\nCross Validation Scores:", cv_scores)
print("Average CV Accuracy:", np.mean(cv_scores))
```

```
======================================================================
MODEL IMPROVEMENT USING GRID SEARCH AND CROSS VALIDATION
======================================================================

Testing Parameters: {'dropout_rate': 0.2, 'learning_rate': 0.001, 'lstm_units': 64}
Accuracy: 0.8385495543479919

Testing Parameters: {'dropout_rate': 0.2, 'learning_rate': 0.001, 'lstm_units': 128}
Accuracy: 0.8417152166366577

Testing Parameters: {'dropout_rate': 0.2, 'learning_rate': 0.0005, 'lstm_units': 64}
Accuracy: 0.8208504319190979

Testing Parameters: {'dropout_rate': 0.2, 'learning_rate': 0.0005, 'lstm_units': 128}
Accuracy: 0.8209223747253418

Testing Parameters: {'dropout_rate': 0.3, 'learning_rate': 0.001, 'lstm_units': 64}
Accuracy: 0.8355996608734131

Testing Parameters: {'dropout_rate': 0.3, 'learning_rate': 0.001, 'lstm_units': 128}
Accuracy: 0.8293402194976807

Testing Parameters: {'dropout_rate': 0.3, 'learning_rate': 0.0005, 'lstm_units': 64}
Accuracy: 0.8296999931335449

Testing Parameters: {'dropout_rate': 0.3, 'learning_rate': 0.0005, 'lstm_units': 128}
Accuracy: 0.8263903856277466

Best Hyperparameters Found:
{'dropout_rate': 0.2, 'learning_rate': 0.001, 'lstm_units': 128}
Best Accuracy: 0.8417152166366577

Cross Validation Scores: [0.8195043802261353, 0.8233897686004639, 0.8214825391769409]
Average CV Accuracy: 0.82145889600118
```

In [ ]: